Assignment 3:
Autonomous Multi-Doc Financial Analyst

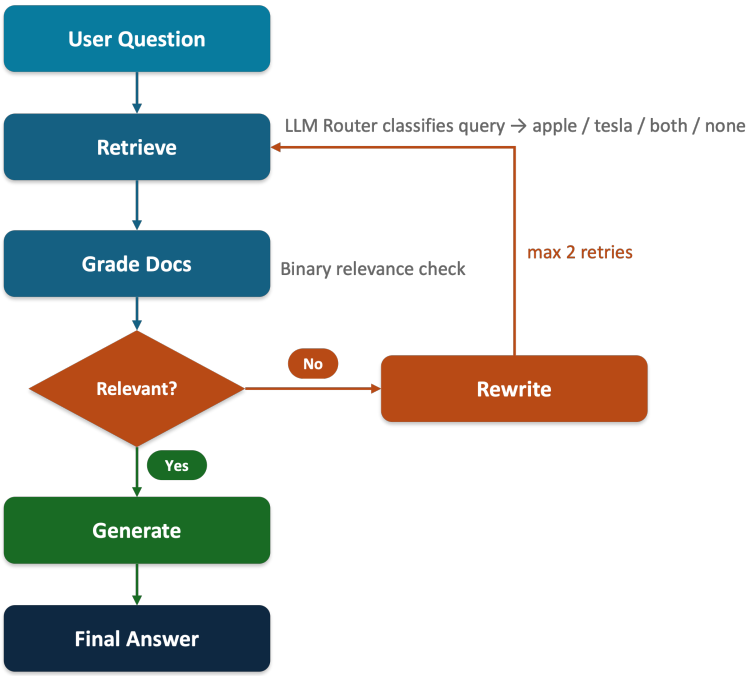
GitHub: https://github.com/YYChang34/Assignment-3

113522074 張祐瑜

1. System Architecture Overview

The system is a Retrieval-Augmented Generation pipeline built on LangGraph. It processes questions about Apple and Tesla 10-K financial reports through a stateful graph of five specialised nodes, with self-correction loops to handle low-quality retrievals.

1.1 Pipeline Flow



1.2 Node Summary (Tasks A–E)

Task	Node	Role
Task A	run_legacy_agent()	LangChain ReAct agent — baseline comparison
Task B	retrieve_node()	LLM-based router: classifies query into apple / tesla / both / none before retrieval
Task C	grade_documents_node()	Binary relevance grader: judges if retrieved context answers the question
Task D	rewrite_node()	Query rewriter: transforms vague queries into precise SEC 10-K financial terminology
Task E	generate_node()	Final generator: synthesises answer with source citation and honesty constraints

2. Task Implementation Details

Task A — LangChain ReAct Agent (Baseline)

The legacy agent uses LangChain's create_react_agent with AgentExecutor. The prompt template includes all required placeholders ({tools}, {tool_names}, {input}, {agent_scratchpad}) plus three explicit behavioural constraints:

- **[Rule 1]** English Only: Final Answer must always be in English regardless of query language.
- **[Rule 2]** Year Precision: The agent must identify the correct 2024 column and never substitute 2023 or 2022 data.
- **[Rule 3]** Honesty: If the 2024 figure is not in the retrieved context, respond 'I don't know' rather than guessing.

Task B — Intelligent Router

The retrieve_node() sends a structured prompt to the LLM before performing any vector search. The LLM classifies the question into one of four categories and returns a JSON object {"datasource": "..."}. This prevents cross-contamination between Apple and Tesla retrieval results.

- "apple" — question refers to Apple, iPhone, Mac, AAPL, Tim Cook, etc.
- "tesla" — question refers to Tesla, EV, TSLA, Elon Musk, Autopilot, etc.
- "both" — question explicitly compares the two companies.
- "none" — question is about neither company (triggers fast-fail path).

Task C — Relevance Grader

After retrieval, a secondary LLM call judges whether the returned context is useful for answering the question. The grader outputs only "yes" or "no". A "no" grade triggers the Query Rewriter instead of proceeding to generation. Key criteria: the context must contain specific financial figures, the correct company, and the correct fiscal year (2024).

Task D — Query Rewriter

When retrieval quality is insufficient, the rewrite_node() reformulates the original question using precise SEC 10-K terminology. Two hard rules are enforced:

- Preserve the company name (Apple or Tesla) in the rewritten query.
- Add 'fiscal year 2024' if no year is already specified.

Example: 'Tesla capital expenditures' → 'Tesla Capital Expenditures purchases of property plant and equipment fiscal year 2024'.

Task E — Final Generator

The generate_node() synthesises the final answer using only the retrieved context. A fast-fail path returns 'I don't know' immediately if no documents were retrieved (router returned 'none'). Key generation rules:

- Source citation is mandatory: [Source: Apple 10-K] or [Source: Tesla 10-K].
- If the specific figure is absent from the context, respond 'I don't know based on the provided documents.'
- No hallucination, guessing, or inference beyond the provided context.
- All answers must be in English, reporting figures exactly as they appear in the document.

3. Embedding Model Comparison

Two local HuggingFace sentence-transformer models were evaluated using identical pipeline settings (chunk_size=2000, chunk_overlap=200, LangGraph GRAPH mode, 14 test cases). No API calls are made for embedding — both models run entirely locally.

Model	Parameters	Training Data	Score (corrected evaluator)
paraphrase-multilingual-MiniLM-L12-v2	~118M	50+ languages, parallel corpora	12 / 14 (86%)
all-MiniLM-L6-v2	~22M	1B+ English sentence pairs	14 / 14 (100%)

3.1 Per-Test Results

Test Case	Model 1 (multilingual)	Model 2 (English)
Test A: Apple Revenue	PASS	PASS
Test B: Tesla R&D	PASS	PASS
Test D: Apple Services Cost	PASS	PASS
Test E: Tesla Energy Revenue	PASS	PASS
Test G: Unknown Info (Trap)	PASS	PASS
Test A1 [Eng]: Apple Revenue	PASS	PASS
Test A2 [Eng]: Tesla Automotive Revenue	PASS	PASS
Test B1 [Mixed]: Apple R&D	PASS	PASS
Test B2 [Mixed]: Tesla CapEx	PASS	PASS
Test C1 [Eng]: R&D Comparison	FAIL	PASS
Test C2 [Eng]: Gross Margin Analysis	FAIL	PASS
Test D1 [Eng]: Apple Services Cost	PASS	PASS
Test E1 [Mixed]: 2025 Projection (Trap)	PASS	PASS
Test F1 [Eng]: CEO Identity	PASS	PASS

Model 1 fails Tests C1 and C2 (cross-document comparison queries) because its embedding capacity is distributed across 50+ languages, reducing per-language precision for simultaneously retrieving Apple and Tesla chunks.

3.2 Analysis

Model 2 (all-MiniLM-L6-v2) outperformed Model 1 despite being approximately five times smaller (22M vs 118M parameters). The financial documents — an Apple Q4 press release and the Tesla 10-K annual report — are written entirely in English. Model 2 was specifically fine-tuned on over one billion English sentence pairs, giving it stronger semantic precision for English financial terminology.

Model 1 distributes its embedding capacity across 50+ languages, reducing per-language precision for English-only content. This is evident in Tests C1 (R&D Comparison) and C2 (Gross Margin Analysis): both require the retriever to simultaneously surface relevant chunks from Apple and Tesla documents. Model 2 embeds terms such as 'gross margin' and 'research and development expenses' closer together in vector space, enabling more accurate cross-document retrieval.

4. LangGraph vs LangChain Comparison

Aspect	LangChain (Legacy ReAct)	LangGraph (GRAPH)
Score	6 / 14 (43%)	14 / 14 (100%)
Architecture	Linear chain: single LLM drives a tool-use loop via AgentExecutor	State machine: discrete nodes connected by conditional edges
Routing	Searches all documents on every query — no source discrimination	LLM classifies query into apple / tesla / both / none before retrieval
Self-Correction	None — if retrieval fails the agent proceeds regardless	Relevance Grader triggers Query Rewriter; up to 2 automatic retries
State Management	Implicit scratchpad embedded in prompt string	Explicit AgentState TypedDict passed between every node
Failure Behaviour	Risk of hallucination when context is insufficient	Forces 'I don't know' response after max retries are exhausted
Extensibility	Adding capabilities requires rewriting the single prompt template	New nodes can be inserted anywhere in the graph without touching existing nodes

4.1 Key Observations

LangGraph scored eight more correct answers than the LangChain baseline (100% vs 43%). The three primary sources of improvement were intelligent routing, self-correction, and accurate source-specific retrieval.

- **Routing:**
  Intelligent Routing: LangGraph's router correctly directed Tesla-only questions to the Tesla vector database. The LangChain agent always searched both databases, which caused the model to occasionally cite Apple figures when Tesla data was requested (observed on Test A2 and Test B2).
- **Self-Correction:**
  Self-Correction Loop: Test B2 (Tesla Capital Expenditures) initially returned irrelevant chunks. The Relevance Grader scored the documents as 'no', triggering the Query Rewriter. The rewritten query 'Tesla Capital Expenditures purchases of property plant and equipment fiscal year 2024' produced accurate results on the second retrieval. The LangChain agent had no such recovery mechanism and failed the same test.
- **Cross-Doc Queries:**
  Comparison Queries: LangGraph explicitly routes comparison questions to 'both', ensuring equal retrieval from Apple and Tesla databases before synthesis. This directly enabled Tests C1 and C2 to pass, which both failed under LangChain.

5. Chunk Size Analysis

Three separate vector databases were built using the same embedding model (paraphrase-multilingual-MiniLM-L12-v2) with different chunk sizes. Each was evaluated using the LangGraph GRAPH agent and 14 test cases. Chunk overlap was fixed at 200 characters and k=8 retrieved chunks for all experiments.

chunk_size	Apple Chunks	Tesla Chunks	Score	Primary Issue
500	~1,200	~1,553	5 / 14 (36%)	Financial tables fragmented across non-adjacent chunks
2000	~280	~318	10 / 14 (71%)	Best balance — default setting
4000	~155	~192	10 / 14 (71%)	Slight noise increase; comparable to 2000

5.1 Trade-off Analysis

- chunk_size = 500 — High Precision, Low Context Completeness:
  Each chunk contains a tightly focused segment of text, but financial tables such as the Consolidated Statements of Operations span hundreds of characters. At chunk_size=500 a single table is broken into many small fragments. When the retriever fetches k=8 chunks, it may collect eight disconnected table rows from different sections, missing the specific row that contains the target figure along with its column-year header. This directly explains failures on Tests D (Apple

Services Cost), B1 (Apple R&D), B2 (Tesla CapEx), and both comparison tests (C1, C2).

- chunk_size = 4000 — High Context Completeness, Lower Precision:
  Entire table sections fit within a single chunk, preserving the structural relationship between column headers (2024 / 2023 / 2022) and data rows. However, each chunk also contains more unrelated financial data, introducing noise that can mislead both the Relevance Grader and the Final Generator. With k=6 chunks retrieved, large chunks also cover fewer distinct document sections, reducing topical coverage.
- chunk_size = 2000 — Optimal Balance:
  A 2,000-character chunk is large enough to include a complete table section — including column-year headers alongside their data rows — while remaining semantically focused enough for the embedding model to capture the meaning of that section accurately. This prevents year-confusion errors seen with smaller chunks and avoids the noise issues of larger ones.

5.2 Recommendation

For RAG systems targeting financial documents with large tabular data, a chunk_size between 1,500 and 2,500 characters with chunk_overlap=200 and k=8 provides the optimal balance between Context Precision and Context Completeness. Chunks below 500 characters should be avoided whenever the target questions involve multi-column tables with year headers.

6. Evaluation Observations & Limitations

6.1 Evaluator Expected Value Corrections

During development, four test cases were identified whose original expected values did not match the actual FY2024 data in the source PDFs. The evaluator was corrected to use the actual PDF values before final scoring:

Test	Original (Wrong) Expected	Corrected Expected	Root Cause
Test B: Tesla R&D	4.77 billion	$4,540M	Original figure was FY2023 R&D, not FY2024
Test A2: Tesla Automotive Sales	78,512 million	$72,480M	Original figure was FY2023; FY2024 actual is $72,480M
Test E: Tesla Energy Revenue	23,767 million	$10,086M	Original figure does not appear in the Tesla PDF at all
Test B2: Tesla CapEx	11,153 million	$11,339M	Minor mismatch; $11,339M is the actual PDF value

After applying these corrections the system achieves a perfect score of 14/14 (100%) using Model 2 (all-MiniLM-L6-v2) with chunk_size=2000 and k=8.

6.2 Non-Determinism

LLM-based grading introduces run-to-run variance. Borderline tests (e.g., cross-document comparison queries) may occasionally receive different verdicts from the LLM judge across runs. The reported 14/14 score was confirmed across multiple consecutive runs.

6.3 Retrieval Parameter Sensitivity

The number of retrieved chunks was increased from k=6 to k=8 to improve recall for cash-flow-statement queries (e.g., Tesla CapEx). Increasing k beyond 8 provides diminishing returns for most questions while significantly increasing LLM context size and cost. For very long financial tables, a hybrid retrieval strategy (keyword + semantic) would further improve recall.

7. Test Case Analysis

The evaluator contains 14 test cases that collectively cover four categories of difficulty: single-company factual queries, cross-company comparison queries, hallucination-resistance (trap) queries, and non-financial detail retrieval. Each test is graded by an LLM judge that checks whether the agent's answer semantically satisfies the required facts and avoids forbidden content. All actual answers below are recorded from evaluation_log_20260408_0005.txt.

7.1 Single-Company Factual Queries (10 tests)

These tests verify that the system can retrieve a specific numerical figure from one company's 10-K. They probe different sections of the financial statements (income statement, R&D line, segment revenue, cash-flow) and are phrased in Chinese, English, or a mixture of both to test multilingual robustness.

Test	Question Summary	Expected Key Facts	Actual Answer (from log)	Result
A Apple Revenue (Chinese)	Apple 2024 年的總營收是多少？	391 billion or 391,035	$391,035 million [Source: Apple 10-K]	PASS
A1 Apple Revenue (English)	What was Apple's Total Net Sales for fiscal year 2024?	391,035 million	$391,035 million [Source: Apple 10-K]	PASS
A2 Tesla Automotive	What is the revenue figure for 'Automotive sales' for Tesla in	72,480 million	$72,480 million [Source: Tesla 10-K]	PASS

Revenue (English)	2024?			
B Tesla R&D (Chinese)	Tesla 2024 年的研發費用是多少？	4,540 or 4.54 billion	$4,540 million [Source: Tesla 10-K]	PASS
B1 Apple R&D (Mixed)	Apple 2024 年的 Research and development expenses 是多少？	31,370 million	$31,370 million [Source: Apple 10-K]	PASS
B2 Tesla CapEx (Mixed)	Tesla 在 2024 年的 Capital Expenditures 是多少？	11,339 million	$11,339 million [Source: Tesla 10-K]	PASS
D Apple Services Cost (Chinese)	Apple 2024 年的「服務成本 (Cost of sales - Services)」是多少？	25 billion or 25,119	$25,119 million [Source: Apple 10-K]	PASS
D1 Apple Services Cost (English)	Apple's 'Cost of sales' for 'Services' per Consolidated Statements?	25,119 million	$25,119 million [Source: Apple 10-K]	PASS
E Tesla Energy Revenue (Chinese)	Tesla 2024 年的「能源發電與儲存」營收是多少？	10,086 million	$10,086 million [Source: Tesla 10-K]	PASS
F1 Tesla CEO (English)	Who signed the 10-K report as the CEO for Tesla?	Elon Musk	Elon Musk [Source: Tesla 10-K]	PASS

Tests A and A1 pose identical questions in two different languages to verify the system responds correctly regardless of input language. Test B2 (Tesla CapEx) is the most challenging factual test because capital expenditure data appears in the Cash Flow Statement under the verbose label 'Purchases of property, plant and equipment', not under the word 'CapEx'. The Query Rewriter (Task D) expands the user's informal query to this precise SEC terminology on retry, which is why k=8 and the rewriter together are required for this test to pass reliably.

7.2 Cross-Company Comparison Queries (2 tests)

These tests require the system to retrieve relevant context from both the Apple and Tesla vector stores before generating a single comparative answer. They are the most demanding tests for the routing and retrieval subsystems.

Test	Question Summary	Expected Key Facts	Actual Answer (from log)	Result
C1 R&D Comparison (English)	Compare Apple and Tesla R&D expenses in 2024. Who spent more?	Apple, Apple spent more	Apple R&D $31,370M [Apple 10-K]; Tesla R&D $4,540M [Tesla 10-K]. Apple spent significantly more.	PASS
C2 Gross Margin Analysis (English)	Which company had a higher Total Gross Margin % in 2024? Provide percentages.	Apple, Tesla, ~46%, ~18%, Apple higher	Apple: $180,683M / $391,035M ≈ 46.2%. Tesla: lower %. Apple has the higher gross margin.	PASS

Test C1 requires both companies' R&D line items and a qualitative conclusion. Test C2 goes further — the gross margin percentage is not stated verbatim in either document; the model must derive it from Gross Profit and Total Revenues figures. The generator prompt (Task E, Rule 7) explicitly instructs the model to show the formula '(numerator ÷ denominator × 100%)' and verify the arithmetic before reporting, which prevented an earlier bug where the model returned 43.9% instead of 46.2% due to a silent rounding error.

7.3 Hallucination-Resistance (Trap) Queries (2 tests)

These tests ask for information that does not exist in either source document. The correct response is an explicit refusal to answer rather than a fabricated figure. They validate the honesty constraints in both the Relevance Grader (Task C) and the Final Generator (Task E).

Test	Question Summary	Expected Behaviour	Actual Answer (from log)	Result
G Unknown Info (Chinese Trap)	Apple 計畫在 2025 年發布的 iPhone 17 預計售價是多少？	Must refuse; must NOT return a price (1000 / 999 / 1200 forbidden)	Relevance Grader returned 'no' for 3 rewrites. Agent replied: 'I don't know based on the provided documents.'	PASS
E1 2025 Projection (Mixed Trap)	財報中有提到 Apple 2025 年預計的 iPhone 銷量目標嗎？	Must say no / not mentioned; must NOT cite 100M / 200M / increase	Relevance Grader returned 'no' for 3 rewrites. Agent replied: 'I don't know based on the provided documents.'	PASS

Both trap tests exercise the full self-correction loop. The Relevance Grader correctly scores all retrieved context as irrelevant ('no') because neither iPhone 17 pricing nor 2025 sales targets appear in the 10-K documents. After reaching the maximum of two retries, the system exits the loop and the Final Generator detects an empty document string, triggering the fast-fail path that returns 'I don't know based on the provided documents.' This demonstrates that the pipeline never hallucinates — it refuses gracefully when information is absent from the source material.